Project #2 — Zipf's Law in a scientific collection: answers to the detailed brief

Biomedical Information Retrieval · Name / ID: _____ · Collection: about 1,000 English PubMed abstracts on GLP-1, one unique PMID each. All tables and plots are shown in the demo system.

Parts I–II — Text preprocessing and the vocabulary

Pipeline. *Tokenization* splits text into tokens; *case folding* maps Data/DATA/data to one term; *punctuation handling* merges "data," and "data." but keeps internal marks (GLP-1); *stopwords* (the, of, and) are kept in one version and removed in another; *stemming* (our own Porter algorithm) cuts suffixes, while *lemmatization* would map words to dictionary forms.

Vocabulary. *Tokens* are all occurrences, *unique terms* are the dictionary entries, and average tokens per document is document length. Each step redefines what a "term" is, so all these numbers change.

Parts III–IV — Collection frequency and document frequency

CF(t) is the total number of occurrences of t in the whole collection; ranking terms by CF gives the rank–frequency list used for Zipf. **DF(t)** is the number of documents that contain t at least once. Always DF ≤ CF and DF ≤ N.

Part V — Zipf distribution: rank–frequency and log-log regression

Zipf's law states f(r) ∝ 1/r^k (ideally k ≈ 1); taking logarithms gives log f = log C − k·log r, a straight line. Experiment 1 (linear axes) shows a sharp L-shaped curve; Experiment 2 (log-log) straightens it, and linear regression log(CF) = a − b·log(r) gives the slope, intercept, exponent k = b, R^2 and RMSE.

Q1 Does the collection follow Zipf's Law? Approximately: the log-log plot is close to a straight line (few very frequent terms, many rare ones), but the top bends away and the tail forms steps, because many rare terms share the same counts (1, 2, 3 …).

Q2 What is the estimated Zipf exponent? The negative slope, about 1.25 here, steeper than the ideal 1; it depends on preprocessing and on the ranks fitted.

Q3 Is R^2 sufficient? No. Sorted ranks make any decreasing curve score a high R^2; the points are not independent; the many tied tail points dominate the fit; and R^2 does not compare against other distributions (e.g. log-normal). It describes a trend, it is not a test.

Q4 Which portion fits best? The high-frequency head, with k near 1. The middle is steeper, and the low-frequency tail fits worst because its tiny integer counts form flat steps.

Part VI — How does preprocessing affect the Zipf distribution?

1 Vocabulary size. Punctuation removal and stemming merge variants and shrink the vocabulary; stopword removal deletes only a short list of types.

2 High-frequency terms. With stopwords the head is function words (and, the, of); without them it shows the subject (glp-1, receptor, patients), and many tokens disappear.

3 Zipf exponent. Each step changes k: merging variants shortens the tail and steepens the slope; removing stopwords flattens the head.

4 Shape. The heavy tail survives every condition; stopwords reshape the head, punctuation and stemming the tail.

Parts VII–VIII — CF vs DF, and the connection to TF-IDF

VII Q1 Why can a term have high CF but relatively low DF? CF counts every occurrence, DF each document once; a term repeated inside a few documents (e.g. an abbreviation reused in one paper) gains CF without spreading.

VII Q2 Which measure shows whether a term is widely distributed? DF (or DF/N); CF cannot tell one document from many.

VII Q3 Why are CF and DF both important? CF reflects volume (tokens, positional postings); DF is the posting-list length and the basis of IDF, i.e. how well a term separates documents.

VIII Why do very common terms get lower IDF? idf = log(N/df): as df approaches N the ratio approaches 1 and its logarithm 0, so a word found everywhere gets almost no weight, while a word in one document gets the maximum log N.

VIII Zipf and TF-IDF. Zipf's frequent head terms are also high-DF terms, so IDF moves weight from the head to rarer, more specific terms; TF-IDF counteracts the skew Zipf describes. IDF must come from DF, not from a term's rank.

Part IX — Why does Zipf's Law matter to an Information Retrieval system? (329 words)

Zipf's law matters to an IR system because it describes how unevenly both the workload and the information are spread over a vocabulary: a small head of terms produces most of the occurrences, while a long tail of rare terms makes up most of the vocabulary.

Stopword removal. The head consists of function words that occur in nearly every document. A term that appears in almost all documents cannot separate relevant from non-relevant ones, so its discriminative value is close to zero, while it costs a large share of the processing and index space. Removing or down-weighting such words therefore saves much

space for little loss. The price is that phrase and negation queries ("not effective", "to be or not to be") need the original text or a positional index.

Inverted index. Zipf shapes every part of the index. The dictionary is dominated by the tail: thousands of terms that occur once each need an entry but have a single posting. The postings are dominated by the head: a few terms have lists that cover almost every document and hold most of the positions. Index storage therefore grows with the head's long posting lists and the tail's large dictionary. Compression exploits exactly this skew: long, dense posting lists compress well with gap (delta) encoding and variable-byte or Elias codes, and front coding or blocking reduces a dictionary of many rare strings.

TF-IDF. Ranking by raw term frequency would reward the head terms that every document shares. IDF gives those terms weights near zero and gives rare, specific terms the largest weights, which is why high-frequency terms have low discriminative power. Zipf explains why such a correction is needed for almost every query.

In short, Zipf tells the designer where the volume is (the head) and where the vocabulary is (the tail), and an IR system treats the two differently: it filters or down-weights the head, compresses its long lists, and keeps the tail cheap to store but highly weighted.

Optional challenge — Does domain specificity affect the Zipf distribution?

A medical collection (GLP-1, PubMed) and a computer-science collection (500 arXiv abstracts from ten subfields) of equal size keep the same heavy-tailed, near-linear log-log shape with similar exponents, but share little vocabulary and none of their top content terms: domain changes which terms form the head, not the form of the law.

Final challenge — Why is an approximate law still useful for IR?

Zipf is not exact: k is not 1, it shifts with preprocessing, and both ends of the curve bend. Yet IR design depends on the shape, not the exact exponent: in every condition and domain a few terms dominate the tokens and a long tail dominates the vocabulary. That robust skew is enough to justify stopword handling, index compression and DF-based weighting, which are then verified on the real system.

Research questions RQ1–RQ5

RQ1 Does the distribution follow Zipf? Approximately; near-linear in log-log, not exact.

RQ2 Empirical exponent? About 1.25, depending on preprocessing and the fitted range.

RQ3 Effect of preprocessing? Stopwords reshape the head; punctuation handling and stemming shrink the tail; k changes.

RQ4 CF vs DF? CF ≥ DF; CF measures volume, DF spread; only DF determines IDF.

RQ5 Why does Zipf matter to IR? It explains stopword removal, index size and compression, and IDF weighting.